

PROJECT 01

House Price Prediction & Gradio Application

End-to-End Regression Project

Full Stack AI & Data Science

1 Project Overview

You will build a complete machine learning project from data exploration to a working Gradio application:

1. Explore the USA Housing dataset (EDA).
2. Train and compare multiple regression models.
3. Build and test a Gradio interface locally.
4. Document your work on GitHub.

2 Dataset

The dataset contains the following columns:

- ▶ Avg. Area Income – Average income of residents in the area
- ▶ Avg. Area House Age – Average age of houses in the area
- ▶ Avg. Area Number of Rooms – Average number of rooms in the area
- ▶ Avg. Area Number of Bedrooms – Average number of bedrooms in the area
- ▶ Area Population – Population of the area
- ▶ Price – Price that the house sold at (target variable)
- ▶ Address – Address of the house

Download: Use the attached `usa_housing.csv` file.

Feature Selection

Do not use **Address** as an input feature. After performing EDA, you may also exclude numerical features that are not useful for your model. Explain your final feature choices.

3 Project Structure

Your GitHub repository should have this structure:

```
house-price-prediction/  
|-- data/  
|   '-- usa_housing.csv  
|-- notebooks/  
|   |-- 1_eda.ipynb  
|   '-- 2_training.ipynb  
|-- app.py  
|-- models/  
|   '-- best_model.pkl  
|-- screenshots/  
|   '-- gradio_interface.png  
|-- README.md  
'-- requirements.txt
```

4 Deliverables

1. **GitHub Repository** – All code must be in a GitHub repository.
2. **1_eda.ipynb** – Exploratory Data Analysis notebook.
3. **2_training.ipynb** – Model training and comparison notebook.
4. **app.py** – Gradio application file.
5. **models/** – Saved pipeline.
6. **README.md** – Must include:
 - ▶ Project overview
 - ▶ Dataset description
 - ▶ Model comparison table with all trained models, R^2 , and MSE
 - ▶ Final model details: name, R^2 score, and why you chose it
 - ▶ Screenshots of your Gradio web interface
 - ▶ Public Hugging Face Space URL, if deployed (optional)
 - ▶ How to run the project
7. **requirements.txt** – List of all Python packages used.

5 Part 1: Exploratory Data Analysis (1_eda.ipynb)

Your EDA notebook must include:

1. **Load and inspect data** – Display basic information such as shape, data types, and first few rows. Check for missing values and duplicates.
2. **Target variable analysis** – Analyze the distribution of house prices using a histogram and statistical summary including mean, median, minimum, maximum, and standard deviation.
3. **Feature distributions** – Visualize the distribution of all numerical features.
4. **Correlation analysis** – Create a correlation heatmap showing relationships between numerical features and identify which features correlate most with **Price**.
5. **Scatter plots** – Plot **Price** against each numerical feature to understand their relationships.
6. **Key findings** – Write your observations:
 - ▶ Which features appear most important?
 - ▶ Are the relationships linear or non-linear?
 - ▶ Which features will you use for modeling?
 - ▶ Which model types might work best?

6 Part 2: Model Training (2_training.ipynb)

Your training notebook must include:

1. **Load data and split** – Load the dataset, remove **Address** from the input features, select any additional features based on your EDA findings, and create training, validation, and test sets. Use a fixed random state so your results are reproducible.
2. **Build safe pipelines** – Keep every preprocessing step required by a model inside its scikit-learn **Pipeline**. Fit preprocessing using the training data only. Do not fit or refit preprocessing on the validation or test data.
3. **Train multiple models** – Train at least 3 different models. Choose from:
 - ▶ Linear Regression

- ▶ Polynomial Regression
- ▶ Ridge
- ▶ Lasso
- ▶ KNN

Experiment with different hyperparameters where applicable.

4. **Model comparison table** – Create a table comparing all models. Include:

- ▶ Model name
- ▶ Train R^2
- ▶ Validation R^2
- ▶ Validation MSE

Sort the table by Validation R^2 .

5. **Select the final model** – Choose the final model using validation performance. Explain your decision, connect the result to your EDA findings, and check for overfitting by comparing Train R^2 with Validation R^2 .
6. **Final test evaluation** – After the model and hyperparameters are locked, evaluate the selected Pipeline on the test set once. Report Test R^2 and Test MSE. Do not use the test results to choose or tune the model.
7. **Save the fitted Pipeline** – Save the complete fitted Pipeline to `models/best_model.pkl`. The saved object must include all preprocessing required for prediction.

7 Part 3: Gradio Application (app.py)

Your Gradio app must include:

1. **Load the trained pipeline** – Load from `models/best_model.pkl`.
2. **Prediction function** – Take feature inputs and return a formatted price prediction.
3. **Gradio interface** – Include:
 - ▶ Input widgets for each feature used by the model
 - ▶ Clear labels and descriptions
 - ▶ Output showing the predicted price
 - ▶ A title and description
 - ▶ At least 2–3 example houses
4. **Professional UI** – Use appropriate input types such as Number or Slider and include helpful information for users.
5. **Test locally** – Run the app with `demo.launch()` and verify realistic and invalid input values.

Optional Extension: Hugging Face Spaces

You may deploy the Gradio application to Hugging Face Spaces. If you choose to deploy it, upload `app.py`, `models/best_model.pkl`, and `requirements.txt`; test the hosted application in a fresh browser session; and include the public Space URL in your README and submission document. Hosting the application is optional and is not required to complete the project.

8 GitHub Repository Requirements

Your repository must include:

1. **README.md** – Include:
 - ▶ Project title and overview
 - ▶ Dataset description

- ▶ Model comparison table with all trained models
 - ▶ Final model details: name, R^2 , MSE, and why it was chosen
 - ▶ Screenshots of the Gradio interface
 - ▶ Public Hugging Face Space URL, if deployed (optional)
 - ▶ Installation instructions
 - ▶ How to run the project
2. **Clean code** – Code should be well-commented and organized in the proper folders.
 3. **requirements.txt** – Include all necessary packages.

9 README.md Template

Below is a template for your README.md file:

```
# House Price Prediction & Gradio Application

## Overview
[Brief description of the project]

## Dataset
- Source: USA Housing dataset
- Features used: [list the features you selected]
- Target: House Price
- Total samples: [number]

## Model Comparison

| Model | Train R2 | Validation R2 | Validation MSE |
|-----|-----|-----|-----|
| [Model 1] | [score] | [score] | [value] |
| [Model 2] | [score] | [score] | [value] |
| [Model 3] | [score] | [score] | [value] |

## Final Model

**Model:** [selected model]
**Test R2:** [score]
**Test MSE:** [value]

**Why this model?**
[Explain why you chose this model based on performance
and EDA findings]

## Web Application

The interface is built with Gradio and can be run locally.

**Optional live app:** [your Hugging Face Space URL, if deployed]

### Screenshots

![Gradio Interface](screenshots/gradio_interface.png)

## Installation

git clone [your-repo-url]
cd house-price-prediction
pip install -r requirements.txt

## Usage
```

Run the web app:

```
python app.py
```

Project Structure

```
house-price-prediction/  
|-- data/  
|-- notebooks/  
|   |-- 1_eda.ipynb  
|   '-- 2_training.ipynb  
|-- app.py  
|-- models/  
|   '-- best_model.pkl  
|-- screenshots/  
|   '-- gradio_interface.png  
|-- README.md  
'-- requirements.txt
```

Technologies Used

- Python
- Pandas, NumPy, Matplotlib, Seaborn
- Scikit-learn
- Gradio

10 Submission

Submit to Google Classroom:

Submission Requirements

Submit one **txt**, **md**, or **pdf** document containing:

- ▶ Key findings from EDA
- ▶ Features used in your final model
- ▶ Final model name and R^2 score
- ▶ Model comparison table with all models you trained
- ▶ GitHub repository link
- ▶ Public Hugging Face Space URL, if deployed (optional)

Example Submission Content

Example

House Price Prediction Project - [Your Name]
GitHub Repository: <https://github.com/yourusername/house-price-prediction>
Hugging Face Space (Optional): *Add the public URL only if you deployed the application.*

Key Findings from EDA:

- *Write your first important finding.*
- *Write your second important finding.*
- *Write your third important finding.*

Features Used in Final Model:

- *Feature 1*
- *Feature 2*
- *Feature 3*

Model Comparison:

Model	Train R ²	Validation R ²	Validation MSE
Model 1	Score	Score	Value
Model 2	Score	Score	Value
Model 3	Score	Score	Value

Final Model: [Selected model]
Test R²: [Score]
Test MSE: [Value]
Why this model: [Explain why validation performance and your EDA findings support this choice.]

11 Setup Instructions

1. Create a GitHub repository.
 2. Clone it to your local machine.
 3. Create the folder structure shown above.
 4. Download `usa_housing.csv` to the `data/` folder.
 5. Install the required packages:
- ```
pip install pandas numpy matplotlib seaborn scikit-learn gradio joblib
```
6. Start with the EDA notebook.

Good luck!